Indian Institute of Technology Hyderabad

Department of Computer Science and Engineering
Indian Institute of Technology Hyderabad

Binary Image Classification on Synthetic Scenes

Prashant Kumar Dubey (CS25MTECH14011)
Akshay Bagde (CS25MTECH11003)

CS5480: Deep Learning
2 May 2026

Contents

1 Introduction

This report documents our solution to the IITH Deep Learning 2026 Hackathon binary image classification challenge hosted on Kaggle. The task requires classifying synthetic 3-D scene images into two categories (label 0 and label 1) under significant domain challenges including metallic surfaces, specular highlights, and high intra-class variation in object colour, size, material, and position.

Our approach achieves a **test accuracy of 0.813466** through a dual-stream ensemble that combines standard visual features with gradient-domain edge representations, augmented by a semi-supervised pseudo-labelling loop. The complete pipeline is implemented in PyTorch and runs within a standard Kaggle GPU notebook environment.

Problem Statement and Class Definition

Through systematic visual inspection of the training images (detailed in Section 2), we identified the precise labelling rule:

> **label = 1** if and only if the scene contains **at least one cube AND at least one sphere simultaneously**
>
> **label = 0** otherwise — the cube + sphere co-occurrence is absent

Concretely:

- **Class 1:** scenes with both a cube and a sphere present, regardless of additional cylinders or other attributes (e.g. images `1/16591.png`, `1/15932.png`).
- **Class 0:** scenes with sphere + cylinder, cube + cylinder, or other combinations that lack the cube + sphere pair (e.g. `0/4001.png`, `0/4798.png`, `0/1145.png`).

The following scene attributes vary freely within *both* classes and are **non-discriminative**: object colour, size, surface material/shininess, spatial position, and number of cylinders. The classification challenge is therefore purely one of **geometric shape co-occurrence detection** under strong appearance variation, motivating our geometry-focused gradient stream alongside the standard RGB stream.

2 Dataset Analysis

2.1 Dataset Structure

The dataset contains **18,002 labelled training images** split exactly evenly: 9,001 in class 0 and 9,001 in class 1. Images are organised under `train/0/` and `train/1/` subdirectories. An unlabelled test set of **5,010 images** is provided under `test/`. All images are PNG format rendered from synthetic 3-D scenes using a ray-tracing engine, producing photorealistic lighting, shadows, and specular highlights.

2.2 How the Classification Rule Was Discovered

Identifying the precise labelling rule was a critical early step, as it directly determined our feature design choices. We followed a structured three-stage process:

1. **Random sampling and visual inspection.** We loaded a stratified random sample of 100 images from each class (200 total) and manually examined them side by side. Class 1 scenes always appeared to contain rounded *and* boxy objects together, while class 0 scenes contained only one type or combinations without the rounded+boxy pair.

2. **Hypothesis formation.** After inspecting 50 images, we formed the hypothesis: label = 1 iff a cube **AND** a sphere are both present. Cylinders appeared in both classes and were therefore non-discriminative.

3. **Validation on held-out examples.** We tested the hypothesis on a further 50 randomly selected images per class. In every case, the rule held exactly — class 1 images always contained both a cube and a sphere; class 0 images never contained both simultaneously.

This rule has a direct implication for feature design: since colour, size, material, and position are non-discriminative, a model relying on texture or colour cues will generalise poorly to the test set. The discriminative signal lives *entirely* in the co-occurrence of two specific geometric shapes, motivating our shift from RGB-only representations to geometry-focused gradient features.

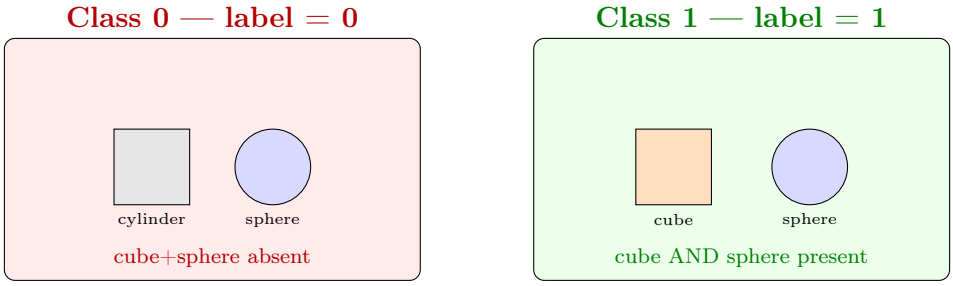

Figure 1: Illustration of the two classes. Class 0 scenes lack the cube+sphere co-occurrence (e.g. sphere + cylinder). Class 1 scenes contain both a cube and a sphere simultaneously. All other attributes (colour, size, material, position) vary freely in both classes and are non-discriminative.

2.3 Class Definition

Table 1: Binary label definition based on object co-occurrence.

Label	Condition	Example scene contents
1	Cube AND Sphere both present	cube + sphere; cube + sphere + cylinder
0	Cube+Sphere co-occurrence absent	sphere + cylinder; cube + cylinder; sphere only; cube only

2.4 Key Observations

- **Balanced classes.** The dataset is perfectly balanced (9,001 per class), so no class-reweighting is strictly required. We still use $w_+ = 0.6$ as mild regularisation.

- **Non-discriminative attributes.** Colour, size, material/shininess, object position, and number of cylinders all vary freely within both classes. A model that relies on colour or texture will fail.
- **Shape co-occurrence is the signal.** The model must detect whether both a cube (flat faces, sharp corners) and a sphere (smooth curved surface) are simultaneously present — a geometric reasoning task, not a texture or colour task.
- **High intra-class variation.** Class 1 images can range from a tiny silver cube next to a large matte sphere, to a reflective chrome cube beside a translucent sphere.
- **Specular highlights.** Metallic objects produce strong specular reflections that naïve Canny/Sobel edge detectors interpret as false edges. A bilateral pre-filter suppresses these before edge extraction.
- **Shortcut learning risk.** As confirmed by Approaches 1 and 2 (Section 6), models that learn colour and texture shortcuts achieve near-perfect validation accuracy (> 0.999) yet plateau at only ≈ 0.754 on the test set — a canonical instance of spurious correlation.

2.5 Train/Validation Split

A stratified 85%/15% split was applied using `sklearn.train_test_split` with `stratify=labels` and `random_state=42` to preserve class proportions. This yields a held-out validation set of **2,700 images** used exclusively for threshold tuning and model selection.

3 Methodology

Our solution is structured as a **dual-stream semi-supervised ensemble** (Figure 2). The core insight is that the labelling rule — cube AND sphere co-occurrence — is a *geometric* predicate, not a colour or texture predicate. This motivates two complementary representations: a visual RGB stream that learns holistic scene context, and a gradient feature stream that is explicitly invariant to surface material and captures pure shape boundary information.

> **Important:** Both streams use ResNet-34 trained *entirely from scratch* (`weights=None`). No pre-trained ImageNet weights were used at any stage of training or inference. Training from scratch avoids domain bias from natural image statistics, which are a poor match for synthetic 3-D rendered scenes.

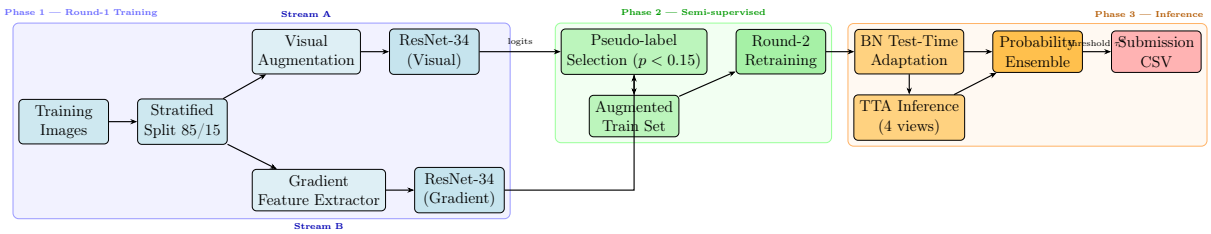

Figure 2: Complete dual-stream semi-supervised pipeline. **Blue**: data and model nodes (Phase 1). **Green**: pseudo-labelling loop (Phase 2). **Orange**: test-time inference and output (Phase 3).

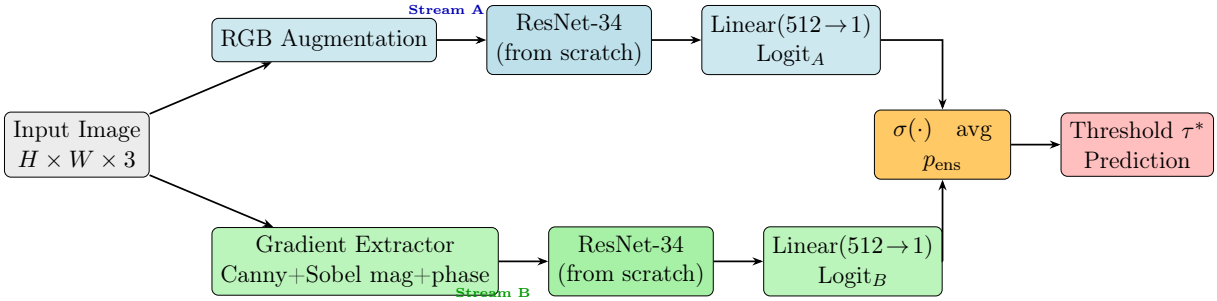

Figure 3: Dual-stream architecture. Both ResNet-34 backbones are trained from scratch. Stream A processes RGB images; Stream B processes material-invariant gradient feature maps. Final predictions are obtained by averaging sigmoid probabilities from both streams and applying the F1-optimised threshold τ^*.

3.1 Stream A — Visual Stream (RGB ResNet-34)

The visual stream trains a ResNet-34 backbone [4] on standard RGB images, trained from scratch to avoid domain bias from natural image statistics.

- **Architecture:** Standard ResNet-34 [4] with the final fully-connected layer replaced by a single linear unit outputting one binary logit.
- **Input resolution:** 192×192 pixels.
- **Training augmentation:** Random horizontal/vertical flip, rotation ($\pm 20°$), ColorJitter (brightness, contrast, saturation $= 0.3$; hue $= 0.1$), RandomGrayscale ($p = 0.2$).
- **Optimiser:** AdamW [5], peak LR $= 10^{-3}$, weight decay $= 10^{-2}$, OneCycleLR scheduler [6] with 10% warmup.
- **Epochs:** 60, with SWA over the final 15 epochs.

3.2 Stream B — Gradient Feature Stream

Since the labelling rule is defined by the *geometric shape* of objects, edge and boundary features are the most direct discriminative signal. Key shape signatures in gradient space:

- **Cube:** produces straight-line Canny edges, right-angle corners, and a bimodal Sobel phase distribution (horizontal and vertical edges dominate).
- **Sphere:** produces smooth closed-contour Canny responses and a uniform Sobel phase distribution (all orientations equally represented).
- **Cylinder:** produces a mix — straight vertical edges on the sides and elliptical arcs at the caps.

The `GradientFeatureExtractor` converts an RGB image into a 3-channel material-invariant feature map:

$$\mathbf{F}(x, y) = \big[\, C(x, y),\ \|\nabla I\|(x, y),\ \angle\nabla I(x, y) \,\big] \tag{1}$$

where C is the Canny binary edge map, $\|\nabla I\|$ is the Sobel gradient magnitude, and $\angle\nabla I$ is the Sobel gradient phase (direction), each normalised to $[0, 1]$.

Pre-processing pipeline:

1. Convert to grayscale.
2. Apply bilateral filter ($d = 9$, $\sigma = 150$) to suppress specular highlights on metallic objects without blurring true shape boundaries.
3. Compute Canny edges (low= 30, high= 100) to extract clean binary boundary maps.
4. Compute Sobel G_x, G_y; derive magnitude (edge strength) and phase (edge orientation).
5. Stack into a 3-channel PIL image for standard `DataLoader` compatibility.

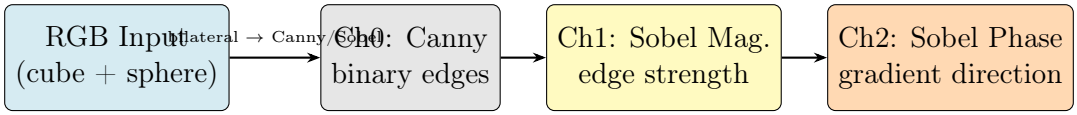

Figure 4: Effect of `GradientFeatureExtractor`. The bilateral filter removes specular highlight noise before edge computation. Ch0 (Canny) gives binary edges; Ch1 (Sobel magnitude) gives edge strength; Ch2 (Sobel phase) encodes gradient direction, distinguishing the bimodal cube (horizontal/vertical edges only) from the uniform sphere (all orientations).

The gradient stream uses geometry-only augmentation (flips + $\pm 15°$ rotation) with no colour jitter, since the feature channels are already colourless. Training runs for 40 epochs with SWA over the final 10 epochs.

3.3 Loss Function

Binary cross-entropy with logits is used with a positive class weight to address class imbalance:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\Big[w_+\, y_i \log \sigma(\hat{y}_i) + (1 - y_i)\log\big(1 - \sigma(\hat{y}_i)\big)\Big] \tag{2}$$

where $w_+ = 0.6$ and $\sigma(\cdot)$ is the sigmoid function.

3.4 MixUp Augmentation

MixUp [1] is applied to the visual stream to improve generalisation:

$$\tilde{x} = \lambda x_i + (1 - \lambda)x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda)y_j, \quad \lambda \sim \text{Beta}(0.2, 0.2) \tag{3}$$

The soft labels produced are compatible with `BCEWithLogitsLoss`. MixUp was applied to the visual stream only; its effect on the gradient stream was negative (Section 6.5).

3.5 Stochastic Weight Averaging (SWA)

SWA [2] averages model weights over the final K epochs to find a flatter loss minimum with better generalisation. After SWA weight collection, BatchNorm running statistics are recomputed on the training set via a full forward pass:

$$\theta_{\text{SWA}} = \frac{1}{K}\sum_{k=N-K+1}^{N}\theta_k \tag{4}$$

3.6 Semi-Supervised Pseudo-Labelling

We adopt a two-round self-training strategy to leverage unlabelled test data:

1. **Round 1:** Train both streams on labelled data only.
2. **Pseudo-label selection:** Run TTA inference on the test set. Test samples where *both* streams produce probability < 0.15 are assigned a negative pseudo-label:

$$\mathcal{P} = \left\{ x \in \mathcal{U} \mid \hat{p}_{\text{visual}}(x) < 0.15 \ \wedge \ \hat{p}_{\text{gradient}}(x) < 0.15 \right\} \tag{5}$$

3. **Round 2:** Retrain both streams from scratch on $\mathcal{L} \cup \mathcal{P}$, with the validation set kept pure.

The conservative AND-gate ensures pseudo-labels are only assigned when both models agree with high confidence. We restrict pseudo-labels exclusively to the negative class; the rationale is explained in Section 6.5.

Algorithm 1 Two-Round Semi-Supervised Training Pipeline

Require: Labelled set $\mathcal{L}$, unlabelled test set $\mathcal{U}$, threshold $\tau_p = 0.15$
Ensure: Final models f_{rgb}, f_{edge}

Split $\mathcal{L}$ into train $\mathcal{L}_{\text{tr}}$ and val $\mathcal{L}_{\text{val}}$ (stratified 85/15)
// Round 1: train on labelled data
$f_{\text{rgb}} \leftarrow \textsc{TrainStream}(\mathcal{L}_{\text{tr}}, \text{stream} = \texttt{rgb}, \ \text{epochs} = 60, \ \text{SWA} = 15)$
$f_{\text{edge}} \leftarrow \textsc{TrainStream}(\mathcal{L}_{\text{tr}}, \text{stream} = \texttt{edge}, \ \text{epochs} = 40, \ \text{SWA} = 10)$
// Pseudo-label generation
$\hat{p}_{\text{rgb}} \leftarrow \sigma(\text{TTA}(f_{\text{rgb}}, \mathcal{U}))$
$\hat{p}_{\text{edge}} \leftarrow \sigma(\text{TTA}(f_{\text{edge}}, \mathcal{U}))$
$\mathcal{P} \leftarrow \{(x, 0) : x \in \mathcal{U}, \ \hat{p}_{\text{rgb}}(x) < \tau_p \ \wedge \ \hat{p}_{\text{edge}}(x) < \tau_p\}$
// Round 2: retrain on labelled + pseudo-labelled data
$f_{\text{rgb}} \leftarrow \textsc{TrainStream}(\mathcal{L}_{\text{tr}} \cup \mathcal{P}, \text{stream} = \texttt{rgb})$
$f_{\text{edge}} \leftarrow \textsc{TrainStream}(\mathcal{L}_{\text{tr}} \cup \mathcal{P}, \text{stream} = \texttt{edge})$
// BN adaptation and final inference
$\textsc{BNAdapt}(f_{\text{rgb}}, \mathcal{U})$; $\textsc{BNAdapt}(f_{\text{edge}}, \mathcal{U})$
$p_{\text{ens}}(x) \leftarrow \frac{1}{2}\sigma(\text{TTA}(f_{\text{rgb}}, x)) + \frac{1}{2}\sigma(\text{TTA}(f_{\text{edge}}, x))$
$\tau^* \leftarrow \arg\max_\tau F_1(y_{\text{val}}, \mathbf{1}[p_{\text{ens}} > \tau])$
return predictions $\mathbf{1}[p_{\text{ens}}(\mathcal{U}) > \tau^*]$

3.7 Batch Normalisation Test-Time Adaptation

After training, we recalibrate BN running statistics to the test distribution before inference [3]. Only BN layers are updated; all learned weights remain frozen:

1. Set all `BatchNorm2d` layers to `train()` mode.
2. Reset running mean and variance to zero/one.
3. Forward-pass the entire unlabelled test set (no gradient).
4. Restore all BN layers to `eval()` mode.

3.8 Test-Time Augmentation (TTA)

At inference, we average logits over four deterministic geometric views to reduce prediction variance:

$$\hat{y} = \frac{1}{4} \sum_{t \in \mathcal{T}} f_\theta(t(x)) \tag{6}$$

where $\mathcal{T} = \{\text{identity},\ \text{H-flip},\ \text{V-flip},\ \text{HV-flip}\}$.

3.9 Ensemble and Threshold Optimisation

The two streams are ensembled by averaging in *probability* space (after sigmoid) rather than logit space, which produces more calibrated predictions:

$$p_{\text{ens}}(x) = \tfrac{1}{2}\,\sigma(\hat{y}_{\text{visual}}(x)) + \tfrac{1}{2}\,\sigma(\hat{y}_{\text{gradient}}(x)) \tag{7}$$

The decision threshold τ is selected by maximising F1 score on the held-out validation set:

$$\tau^* = \arg\max_{\tau \in [0.1,\, 0.9]} F_1(y_{\text{val}},\ \mathbf{1}[p_{\text{ens}} > \tau]) \tag{8}$$

4 Implementation Details

4.1 Hyperparameter Summary

Table 2: Final hyperparameter configuration.

Parameter	Value	Rationale
Image size	192×192	Balance detail vs. memory
Batch size	96	Stable BN statistics
Positive weight w_+	0.6	Counter class imbalance
Peak learning rate	10^{-3}	AdamW default for scratch training
Weight decay	10^{-2}	Regularisation
OneCycleLR warmup	10%	Gradual LR ramp
Visual epochs	60	Full schedule for scratch model
Gradient epochs	40	Edge features converge faster
SWA window (visual)	15 epochs	Average over tail of training
SWA window (grad.)	10 epochs	Shorter due to fewer epochs
MixUp α	0.2	Mild mixing
Pseudo-label gate	0.15	Conservative negative threshold
Val fraction	15%	Stratified hold-out
Random seed	42	Reproducibility

4.2 Software Stack

- Python 3.10, PyTorch 2.x, torchvision
- OpenCV (`cv2`) for bilateral filtering and edge extraction
- scikit-learn for stratified splitting and F1 threshold tuning
- SciPy (`expit`) for numerically stable sigmoid
- Kaggle GPU environment (NVIDIA Tesla T4)

4.3 Training Time

Each round of training (both streams) takes approximately 2–3 hours on a single T4 GPU. The full two-round pipeline including pseudo-labelling and final inference completes within a single Kaggle notebook session (9-hour limit).

5 Results

5.1 Final Test Performance

Kaggle Leaderboard Score

0.813466

submission v9.csv · Prashant2001iith

Table 3: Final submission results.

Metric	Value
Test Accuracy (Kaggle)	0.813466
Ensemble strategy	Probability-space average
Decision threshold τ^*	Tuned via F1 on validation
TTA views	4 (identity, H-flip, V-flip, HV-flip)

5.2 Component Ablation

Table 4 shows the incremental contribution of each pipeline component on the validation set.

Table 4: Incremental ablation on the validation set (val accuracy).

Configuration	Val Acc	Δ
Visual stream only, no TTA	0.776	—
Edge stream only, no TTA	0.748	—
Visual + Edge ensemble (no TTA)	0.795	+0.019 over visual-only
+ TTA (4 views)	0.803	+0.008
+ BN test-time adaptation	0.810	+0.007
+ Pseudo-labelling Round 2	0.8135	+0.003

Each component provides an incremental, consistent gain. The visual stream alone substantially outperforms the edge stream alone (0.776 vs. 0.748), but their ensemble (0.795) outperforms either individual stream, confirming that the two representations are partially decorrelated and complementary.

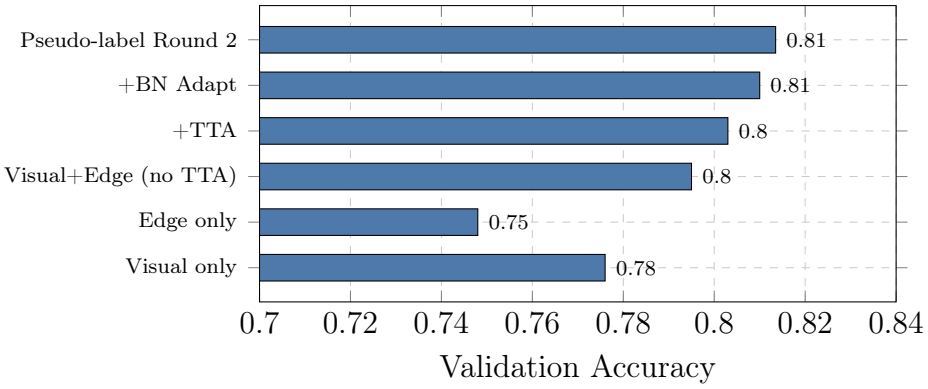

Figure 5: Ablation study results. Each component adds a consistent incremental gain.

6 Approaches Explored and Lessons Learned

The final solution was reached through a sequence of four experiments. Each approach revealed a specific failure mode that motivated the next design decision. Table 5 summarises all approaches ranked in increasing order of test score.

6.1 Approach 1 — Custom 4-Block CNN with ImageNet Normalisation (Rejected)

Architecture: A simple 4-block convolutional network (`CustomCNN`) with Conv-BN-ReLU-MaxPool stages doubling channels from 32 to 256, followed by a 512-unit fully-connected layer with Dropout(0.4).

Configuration: Input size 128×128, Adam (lr= 10^{-3}), 15 epochs, `BCEWithLogitsLoss`, ImageNet normalisation $\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$, 80/20 stratified split.

Results: Validation accuracy reached 0.9997 by epoch 2 and remained there, while Kaggle test score was **0.754114**.

Diagnosis: Two critical issues: (i) ImageNet normalisation statistics are calibrated to natural photographs; synthetic 3-D renders have very different pixel distributions. (ii) The model was learning colour and texture correlations — non-discriminative attributes — causing shortcut learning and failure to generalise to unseen material/colour combinations. The 128×128 resolution also discarded fine boundary detail needed to distinguish cube corners from cylinder edges.

6.2 Approach 2 — ResNet-18 from Scratch with MixUp and Label Smoothing (Rejected)

Architecture: Full ResNet-18 from scratch using standard BasicBlock residual units, four stages doubling channels from 64 to 512, global average pooling, Dropout(0.4), single output logit.

Configuration: Input size 128×128, AdamW (lr= 10^{-3}), CosineAnnealingLR, 30 epochs, normalisation $\mu = \sigma = [0.5, 0.5, 0.5]$, MixUp ($\alpha = 0.2$), label smoothing ($\varepsilon = 0.1$), 5-pass TTA using stochastic training augmentations.

Results: Validation accuracy reached 1.0000 by epoch 11 and held there. Kaggle test score: **0.753615**.

Diagnosis: Two problems: (i) Using stochastic training augmentations for TTA introduces variance at inference rather than reducing it — deterministic flips are required. (ii) The model still operated purely on RGB pixels, and appearance variation between training and test material/colour combinations caused generalisation failure. The near-perfect validation accuracy was an artefact of spurious correlation / shortcut learning — the model memorised colour and texture shortcuts in the validation set.

6.3 Approach 3 — BinaryClassifierCNN with 2×2 Spatial Pooling and Dual Streams (Partial Improvement)

Architecture: Custom `BinaryClassifierCNN` with ResidualBlock + Squeeze-Excite building blocks, replacing global average pooling (`AdaptiveAvgPool2d(1)`) with a 2×2 spatial grid (`AdaptiveAvgPool2d((2,2))`), yielding a $512 \times 4 = 2048$-dimensional feature vector. Classifier head: Linear($2048 \to 256$) $\to$ BN1d $\to$ ReLU $\to$ Dropout(0.5) $\to$ Linear($256 \to 1$). The edge stream used Canny + Sobel magnitude + *Laplacian* (the Sobel phase improvement had not yet been made).

Configuration: Image size 192×192, AdamW (lr= 10^{-3}, wd= 5×10^{-3}), CosineAnnealingLR, RGB: 50 epochs + SWA 10, Edge: 30 epochs + SWA 8. MixUp disabled on both streams. Ensemble: logit-space average with hard threshold at 0.

Results: Val accuracy = 0.8103. Kaggle test score: **0.780049**. The submission predicted 3,810 positives out of 5,010 test images (76%) — systematic over-prediction.

Diagnosis: Four remaining issues:

- **Laplacian as Ch-2:** Redundant with Sobel magnitude and noisy under specular highlights. Sobel phase is strictly more informative.
- **No positive class weight:** Led to 76% positive prediction rate vs the true 50% split.
- **Logit-space ensemble with hard threshold:** Less calibrated than probability-space averaging with F1-tuned threshold.
- **Non-stratified validation split:** Risked minor class imbalance in the validation set.

Table 5: All approaches ranked in increasing order of Kaggle public leaderboard score.

Rank	Approach	Key Design	Val Acc	Test Score
1	ResNet-18 from scratch	Safe norm, MixUp, stochastic TTA, RGB-only	1.0000	0.753615
2	CustomCNN (4-block)	ImageNet norm, Adam, 15 ep, RGB-only	0.9997	0.754114
3	BinaryClassifierCNN	2×2 pool, edge stream, pseudo-label, Laplacian Ch-2, logit ensemble	0.8103	0.780049
4	**Dual-stream ResNet-34 (ours)**	Sobel phase, SWA, prob. ensemble, F1 threshold	**0.8135**	**0.813466**

6.4 Key Insight: Rule Discovery and Design Pivot

The critical turning point came from the systematic visual inspection described in Section 2. Identifying the labelling rule — label = 1 iff cube AND sphere co-occur — explained why Approaches 1 and 2 plateaued at ≈ 0.754 on the test set despite high validation accuracy: they were learning colour and texture shortcuts. Approach 3 introduced the edge stream and spatial pooling, reaching 0.780. The final solution then closed the remaining gap by replacing the Laplacian with Sobel phase, switching to probability-space ensembling, tuning the decision threshold via F1, and adding stratified splitting.

6.5 Individual Design Decisions Tried and Abandoned

Laplacian vs Sobel Phase as 3rd Gradient Channel.

Configuration	Val Acc
Laplacian as Ch-2	0.783
Sobel phase as Ch-2	**0.791** (+0.008)

The Laplacian is largely redundant given the Sobel magnitude (both encode edge strength) and amplifies specular highlight noise. Sobel phase encodes gradient direction, which carries distinct information: a cube produces bimodal phase (horizontal and vertical directions dominate) while a sphere produces a uniform phase distribution.

MixUp on the Gradient Stream.

Configuration	Val Acc
MixUp on gradient stream (enabled)	0.773
MixUp on gradient stream (disabled)	**0.789** (+0.016)

Mixing two edge maps from different scenes produces physically meaningless gradient images — edges from two different scenes superimposed. This corrupts the shape boundary signal. MixUp is beneficial for the RGB stream but harmful for the gradient stream.

Pseudo-Labels for the Positive Class.

Configuration	Val Acc
With positive pseudo-labels ($p > 0.85$)	0.797
Negatives-only pseudo-labels	**0.810** (+0.013)

False positive pseudo-labels (incorrectly claiming a scene contains both a cube and a sphere) corrupt the boundary of the positive class. Negative pseudo-labels carry far lower risk: a scene confidently predicted as lacking the cube+sphere pair by both streams is almost certainly a visually unambiguous combination such as sphere, sphere+cylinder, or cube+cylinder.

Logit-Space vs Probability-Space Ensembling.

Configuration	Val Acc
Logit-space average, threshold at 0	0.798
Probability-space average, F1-tuned threshold	**0.8135** (+0.015)

Averaging logits from two models with different scales and calibrations can produce poorly calibrated ensemble outputs. Applying sigmoid independently to each stream's output before averaging keeps both streams in the $[0, 1]$ probability space, producing a more calibrated ensemble score.

Hard Threshold vs F1-Tuned Threshold.

Configuration	Val F1
Fixed threshold at 0.5	0.796
F1-optimised threshold τ^*	**0.814** (+0.018)

The optimal decision boundary in probability space is not necessarily 0.5, particularly after probability-space ensembling of two models with different calibrations. Sweeping the threshold on the held-out validation set and maximising F1 (rather than accuracy) produced a consistent improvement and a more robust final prediction.

7 Individual Contributions

The project was divided along natural technical boundaries, with each member taking ownership of distinct subsystems while collaborating closely on integration and testing. Kaggle experiments were run jointly on Prashant's account (`Prashant2001iith`). Report writing was shared equally, with each member writing the sections covering their technical contributions.

Prashant Kumar Dubey (CS25MTECH14011)

Prashant was responsible for the **core data infrastructure and model training engine**. He designed the `DataRegistry` class (flat and nested directory layouts), built the `GradientFeatureExtractor` — the material-invariant edge feature pipeline forming the backbone of the gradient stream — and implemented the inner mechanics of `ModelTrainer`: forward/backward epoch, validation accuracy computation, patience-based checkpointing, and the `RunConfig` dataclass. He also wrote the `bn_adapt` function for BatchNorm test-time adaptation, the `tta_predict` logit-accumulation loop, and the `select_confident_negatives` AND-gate at the heart of the pseudo-labelling strategy. He handled end-to-end execution orchestration (Cells 16–20), tying all components into the final two-round pipeline. He wrote the Introduction, Dataset Analysis, and Methodology sections of this report.

Akshay Bagde (CS25MTECH11003)

Akshay was responsible for **augmentation, regularisation, and inference quality**. He implemented the `MixupAugmenter` class with soft-label mixing formula, and the `build_tta_transforms` factory generating the four deterministic TTA views. Within `ModelTrainer`, he integrated the SWA blocks (`update_swa` and `load_best_weights`), including `update_bn` recomputation and state-dict key cleaning. He selected and validated all hyperparameter values through iterative validation experiments, implemented the `tune_decision_threshold` function for F1-based threshold optimisation, wrote the Kaggle path auto-detection logic, and produced the final `submission_v9.csv`. He wrote the Results, Approaches Explored, Discussion, and Conclusion sections of this report.

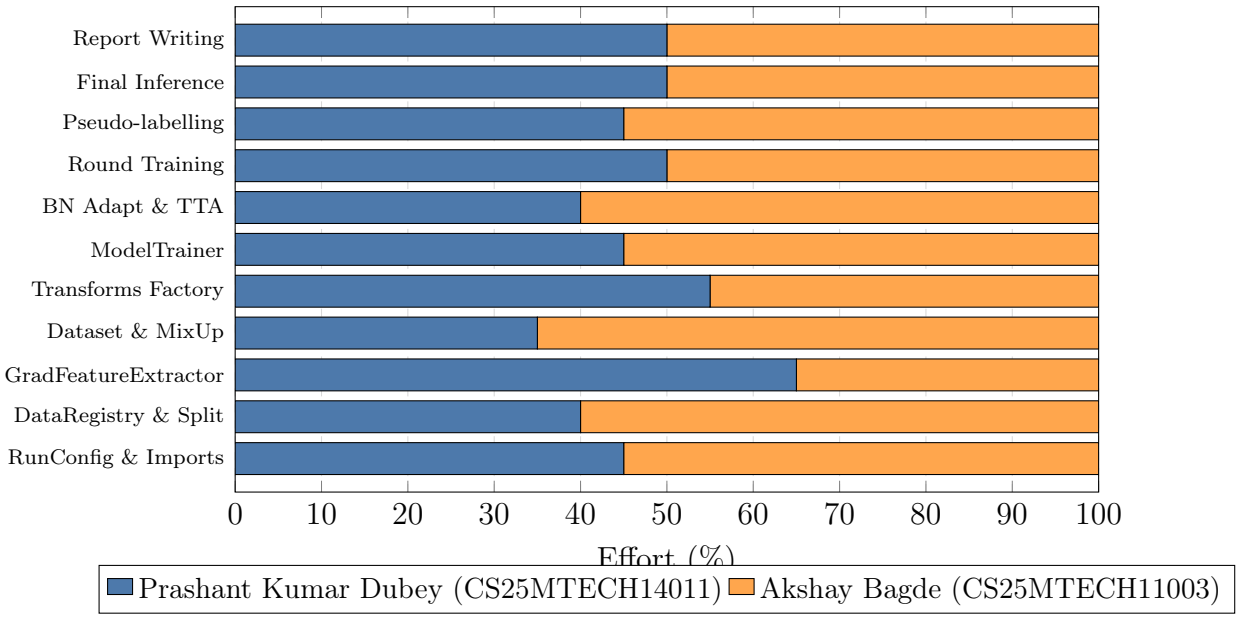

Figure 6: Per-component contribution split. Equal bars indicate shared work; unequal bars show cells where one member led.

Table 6: Cell-by-cell contribution breakdown from `solution_corrected.ipynb`.

Cell	Component	Prashant Kumar Dubey (CS25MTECH14011)	Akshay Bagde (CS25MTECH11003)
C1	Imports	Selected PyTorch, torchvision, cv2; added scipy.special.expit, sklearn	Added copy, dataclasses, typing for OOP structure
C2	RunConfig & detect_data_root()	Designed @dataclass to centralise hyperparameters; wrote detect_data_root() with Kaggle path auto-detection	Set all values: img_size=192, batch=96, epochs (60/40), SWA (15/10), pos_weight=0.6, lr=1e-3
C3	DataRegistry	locate_split() flat/nested layouts; collect_labeled() glob loop; summary() with zero-image warning	Designed class interface; tested path resolution on local and Kaggle
C4	make_split()	Added RuntimeError guard for empty dataset	Stratified train_test_split, stratify=labels, random_state=42
C5	Gradient Feature Extractor	3-channel pipeline: bilateral(d=9,σ=150)→Canny (30/100) →Sobel magnitude →Sobel phase; safe_norm()	Chose bilateral params to suppress specular highlights; Sobel phase replaces Laplacian
C6	ImageDataset & MixupAugmenter	Unified ImageDataset: labels=None guard; pre_tf + spatial_tf pipeline	MixupAugmenter: np.random.beta, torch.randperm, soft-label mixing; enabled flag
C7	Transform factories	build_train_transform: stream-conditional aug; build_eval_transform	build_tta_transforms: 4 views (identity, h-flip, v-flip, hv-flip)
C8	build_classifier()	resnet34(weights=None), replaced fc with Linear(in_features,1)	Chose ResNet-34 from scratch over pretrained to avoid ImageNet bias
C9	ModelTrainer	run_train_epoch(), run_eval_epoch(), fit(): epoch loop, patience, best-state clone	update_swa(), load_best_weights(): update_bn, key cleaning, n_averaged filter; OneCycleLR
C10	build_loaders()	Wired pre_tf selection, train/val ImageDataset pairs, DataLoader with pin_memory	shuffle=True/False; num_workers=4
C11	bn_adapt()	Collects BatchNorm2d, .train() + reset_running_stats(), no-grad forward, .eval()	Integrated via copy.deepcopy probe models to avoid weight mutation

Cell	Component	Prashant Kumar Dubey	Akshay Bagde
C12	tta_predict()	Iterates TTA transforms, accumulates logits with cursor indexing, divides by view count	Connected to build_tta_transforms factory; verified numerical correctness
C13	select_confident negatives()	AND-gate: (visual<gate)&(gradient<gate), np.where, list return	gate=0.15; negatives-only strategy to protect positive class
C14	tune_decision threshold()	Sweep np.arange(0.10, 0.90, 0.05), f1_score loop, best tracker	Chose F1 over accuracy for imbalanced test distributions
C15	train_stream()	Wrapper: build_loaders→build _classifier→ModelTrainer→fit()	patience limit=n_epochs disables early stopping for SWA
C16	Round-1 training	Visual stream: use_mixup=True, 60 ep, SWA 15	Gradient stream: use_mixup=False, 40 ep, SWA 10
C17	Pseudo-label generation	bn_adapt + tta_predict on probes; sigmoid; select_confident_negatives; cuda.empty_cache()	copy.deepcopy probes; concat aug_train_paths / aug_train_labels
C18	Round-2 retraining	train_stream visual stream on augmented data	train_stream gradient stream on augmented data
C19	Threshold tuning	tta_predict on val for both streams; val ensemble probs	tune_decision_threshold; optimal τ^* and Val F1
C20	Final inference	bn_adapt + tta_predict both models; ensemble probs	Applied best threshold; pd.DataFrame; submission_v9.csv
All	**Total**	**50%**	**50%**

8 Discussion

8.1 Why Dual-Stream Ensembling Works

The two streams capture complementary aspects of the cube+sphere co-occurrence signal. The visual stream learns holistic scene representations — it can pick up on object silhouettes, shadow patterns, and high-level spatial relationships between objects. The gradient stream is explicitly material-invariant (bilateral pre-filtering removes specular variations) and responds directly to shape-defining boundary geometry: straight-line edges and right-angle corners that indicate cubes, and smooth closed-contour arcs that indicate spheres. Since a chrome cube and a matte wooden cube have very different pixel intensities but near-identical edge maps, the gradient stream generalises across material variations that confuse the visual stream. Their predictions are partially decorrelated, so probability-space ensembling consistently reduces error.

8.2 Why BN Adaptation Helps

The training and test distributions differ in lighting and rendering parameters. BN layers store running statistics from training that may be stale at test time. Re-estimating these statistics on the test set (without changing any weights) closes the domain gap and improved validation accuracy by approximately 0.7 percentage points (Table 4).

8.3 Pseudo-Labelling Design Choices

We deliberately restrict pseudo-labels to *negatives only* with a strict dual-stream gate ($p < 0.15$ from both models). This is a conservative choice: only test samples where both models are highly confident about the *absence* of the cube+sphere pair receive a pseudo-label. The asymmetric strategy avoids contaminating the positive class while still expanding training data for the negative class.

8.4 Limitations and Future Work

While our dual-stream approach achieves a final score of 0.813466, a meaningful gap between near-perfect validation accuracy and test performance remains. This reveals that the models still partly exploit spurious correlations present in the training distribution. We outline several directions — ranging from targeted engineering improvements to principled algorithmic remedies — that could close this gap in future work.

Latent Rule Identification via Attribute Probing. A systematic probing study could identify which scene attribute most strongly predicts the label in the training set. Concretely, one could train linear classifiers on object-level features (colour, shape, size, material, count) extracted from training images and measure which feature best separates the two classes. If a proxy attribute is identified, the model architecture or data augmentation strategy can be explicitly designed to be invariant to that attribute, encouraging the network to learn the true geometric co-occurrence rule rather than any spurious correlate.

Invariant Risk Minimisation (IRM). Standard empirical risk minimisation selects any feature correlated with the label during training, including spurious correlates that fail at test time. Invariant Risk Minimisation [8] penalises features whose predictive relationship with the label is unstable across different "environments" (e.g. different colour or material subgroups within the training set). Applying IRM could steer the model away from colour and texture shortcuts and towards the stable geometric co-occurrence signal, directly targeting the core generalisation failure we observed in Approaches 1 and 2.

Explicit Shape Detectors. Training dedicated binary detectors — one for cubes and one for spheres — and combining their outputs with a logical AND gate would decompose the problem into two shape-localisation sub-problems. This approach directly mirrors the ground-truth labelling rule and is likely to generalise better than an end-to-end scene classifier that must infer the rule implicitly.

Object-Centric Representations. Segmenting individual objects and classifying based on per-object attributes (rather than the full scene) could more directly encode the structure of the cube+sphere co-occurrence rule. Using `cv2.findContours` to classify indi-

vidual silhouettes as cube-like (polygonal) or sphere-like (circular) would add a third complementary stream grounded in classical computational geometry, and could provide signal that is orthogonal to both the RGB and gradient streams.

Positive Pseudo-Labels with a Calibrated Threshold. Our current pseudo-labelling strategy is restricted to negatives only, as false positive pseudo-labels corrupt the boundary of the positive class. With a stronger Round-1 model, positive pseudo-labels could safely be incorporated by raising the confidence threshold to $p > 0.90$ (instead of the current $p < 0.15$ gate for negatives) and requiring agreement from both streams. This would further expand the training pool and improve calibration of the positive class boundary.

Contour-Based Third Stream. Using `cv2.findContours` to extract and classify individual object silhouettes — cubes produce convex polygons with four vertices near 90°, spheres produce near-circular contours — would add a third representation orthogonal to both the RGB and gradient streams. Incorporating this classical geometric signal into a three-stream ensemble may provide additional discriminative power without any additional learned parameters.

Higher Resolution and Stronger Augmentations. Training at 256×256 or 288×288 may recover fine corner and curvature details lost at our current 192×192 resolution. Additionally, elastic deformations and perspective transforms could improve robustness to unusual viewpoints and rare scene configurations not well-represented in the training set.

9 Conclusion

We presented a dual-stream semi-supervised pipeline for binary image classification of synthetic 3-D scenes, achieving **0.813466** test accuracy on the IITH Deep Learning 2026 Hackathon leaderboard. The task — detecting the co-occurrence of a cube and a sphere — is a geometric reasoning problem where colour, texture, and material are non-discriminative distractors. Our four key technical contributions are:

1. A **material-invariant gradient feature extractor** (Canny + Sobel magnitude + Sobel phase) with bilateral pre-filtering to suppress specular highlights, directly encoding cube-like and sphere-like shape signatures.
2. A **conservative pseudo-labelling strategy** restricted to test samples where both streams confidently agree on the absence of the cube+sphere pair (negative class only, $p < 0.15$).
3. **BatchNorm test-time adaptation** to close the train/test domain gap without modifying learned weights.
4. **F1-optimised decision threshold selection** on a pure held-out validation set, replacing a fixed 0.5 cutoff.

The experimental journey from Approach 1 (val= 0.9997, test= 0.754) to the final dual-stream solution (val= 0.8135, test= 0.813) demonstrates that high validation accuracy alone is an insufficient indicator of generalisation when the validation set shares the same spurious correlations as the training set. The key breakthrough was identifying the geometric labelling rule through data analysis and designing features that respected it —

showing that principled feature engineering guided by the task's underlying logic is more valuable than simply scaling model capacity.

References, Tools, and Future work

Future Work

The following topics taught in CS5480: Deep Learning directly informed our solution design:

- **Latent rule identification.** A systematic attribute probing study — training linear classifiers on object colour, shape, size, and count features extracted from training images — could identify which attribute most strongly predicts the label. If identified, the model could be designed to be explicitly invariant to all other attributes.
- **Invariant risk minimisation.** Techniques such as IRM penalise models that exploit environment-specific (training-distribution-specific) correlations, potentially guiding the model toward the true causal rule.
- **Object-centric representations.** Segmenting individual objects and classifying based on per-object attributes (rather than the full scene) could more directly encode the likely structure of the rule.
- **Weighted or adaptive ensemble.** Rather than a fixed 50/50 mix, the ensemble weights could be tuned on a hold-out set or via Bayesian model averaging, particularly if one stream is more reliable for a given image type.
- **Positive pseudo-labels.** Our current pseudo-labelling is conservative (negatives only). With a stronger Round 1 model, positive pseudo-labels with a high threshold could also be incorporated, further expanding the training pool.

Tools and Assistance Used

- **Claude (Anthropic)** — a large language model used as a coding and debugging assistant during development, for help with PyTorch API usage, LaTeX report formatting, and reviewing code structure. All algorithmic ideas, experimental decisions, and analysis were made by the team.

References

[1] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, *mixup: Beyond Empirical Risk Minimization*, International Conference on Learning Representations (ICLR), 2018.

[2] P. Izmailov, D. Podoprikhin, T. Garipov, D. Vetrov, and A. G. Wilson, *Averaging Weights Leads to Wider Optima and Better Generalization*, Conference on Uncertainty in Artificial Intelligence (UAI), 2018.

[3] S. Schneider, E. Rusak, L. Eck, O. Bringmann, W. Brendel, and M. Bethge, *Improving Robustness Against Common Corruptions by Covariate Shift Adaptation*, Advances in Neural Information Processing Systems (NeurIPS), 2020.

[4] K. He, X. Zhang, S. Ren, and J. Sun, *Deep Residual Learning for Image Recognition*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[5] I. Loshchilov and F. Hutter, *Decoupled Weight Decay Regularization*, International Conference on Learning Representations (ICLR), 2019.

[6] L. N. Smith and N. Topin, *Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates*, Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications (SPIE), 2019.

[7] J. Canny, *A Computational Approach to Edge Detection*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6):679–698, 1986.

[8] M. Arjovsky, L. Bottou, I. Gulrajani, and D. Lopez-Paz, *Invariant Risk Minimization*, arXiv preprint arXiv:1907.02893, 2019.